SOFTWARE REQUIREMENTS SPECIFICATION

1) Business story – CoachApp – Komplexní nástroj pro sportovní trenéry

Problém: Práce trenéra v amatérském a poloprofesionálním fotbale obnáší mimo jiné také obrovské množství administrativy. Trenéři se při organizaci často spoléhají i na několik různých nástrojů – od nepřehledných tabulek v Excelu pro statistiky, přes konverzace na WhatsAppu pro řešení omluvenek a docházky, až po papírové bloky a izolované dokumenty pro přípravu tréninků a zápasů. Tento nesystémový přístup může vést ke ztrátě důležitých dat, způsobuje pomalou komunikaci a ztěžuje sledování dlouhodobého vývoje hráčů. Hledání historie zranění, aktuálních kontaktních údajů nebo statistik z minulých sezón je zdlouhavé a neefektivní.
Řešení (Vize produktu): CoachApp je moderní webová aplikace navržená s ohledem na reálné potřeby z trenérské praxe. Slouží jako centralizovaný digitální nástroj, který sjednocuje veškerou agendu kolem týmu do jednoho přehledného a intuitivního rozhraní. Hlavním cílem je zbavit trenéry zbytečného papírování a neustálého dohledávání informací v konverzacích a jednotlivých tabulkách, aby se mohli plně soustředit na to nejdůležitější – tréninkový proces a rozvoj týmu a samotných hráčů.
Klíčové hodnoty a funkce

- Efektivní plánování a správa docházky: Nástroj pro snadné vytvoření tréninkových jednotek či zápasů s možností pro hráče se přihlásit nebo se včas a přehledně omluvit. Trenér má tak okamžitý přehled o počtu účastníků a může adekvátně připravit tréninkový plán či zápasovou soupisku.
- Kompletní správa týmu: Sjednocená evidence hráčů a realizačního týmu, obsahující základní osobní údaje a také fyzické parametry.
- Zdravotní stav a dostupnost: Rychlý přehled o aktuálním zdravotním stavu hráčů (připraven / zraněn), umožňující efektivní plánování tréninkových jednotek a zápasových nominací.
- Historie a analytika: Dlouhodobé sledování herní výkonnosti (góly, asistence, odehrané minuty) napříč sezónami pro objektivní a daty podložené hodnocení hráčů.
- Centralizované kontakty: Snadný a rychlý přístup k e-mailům a telefonním číslům hráčů (případně zákonných zástupců) přímo v aplikaci, což usnadňuje týmovou logistiku.

Cílová skupina: Aplikace je primárně určena pro hlavní trenéry, asistenty trenérů a vedoucí mužstev, kteří chtějí mít svá data strukturovaná, bezpečně uložená a neustále dostupná z jakéhokoliv zařízení. Jejím cílem je nahradit staré nepřehledné metody, které někteří trenéři stále používají. CoachApp uživatelům šetří čas, usnadňuje každodenní rutinní úkoly a díky přehlednému uživatelskému rozhraní výrazně zpříjemňuje a zrychluje celkovou organizaci týmu.

2) Use Case Diagram

UC – REGISTRACE NOVÉHO ČLENA

ID UC: UC-02

Název: Registrovat nového člena

Aktéři: Uživatel s oprávněním správy členů (např. Admin nebo Trenér)

Vstupní podmínka: Uživatel je přihlášen v aplikaci s rolí, která umožňuje zápis a úpravu členů

Main flow:

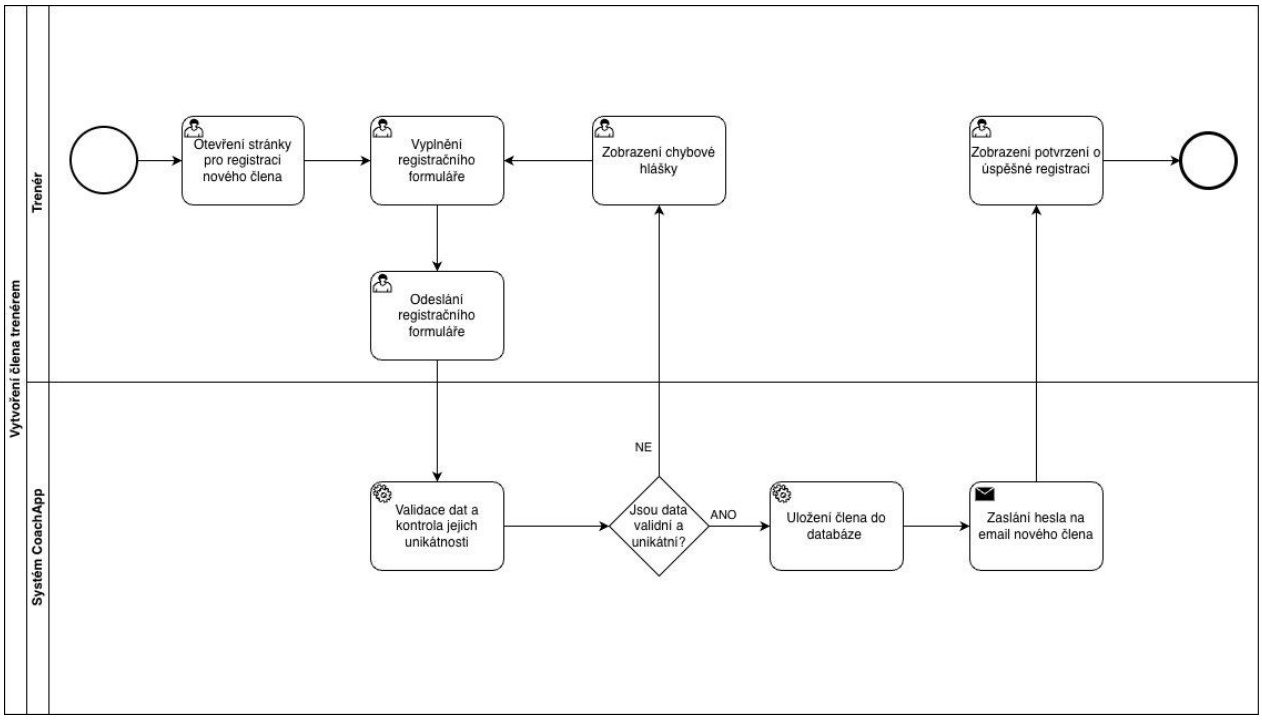
1) Oprávněný uživatel v postranním menu nebo na nástěnce klikne na „Nový člen“.
2) Systém zobrazí registrační formulář s jednotlivými položkami pro základní údaje.
3) Uživatel vybere funkci člena – pokud se jedná o funkci „Hráč“, systém dynamicky zobrazí další položky specifické přímo pro hráče.

4) Uživatel vyplní povinná pole (jméno, příjmení, datum narození, e-mail, město) a případné hráčské parametry (FAČR ID, výška, váha, preferovaná noha a pozice).
5) Uživatel klikne na políčko „PŘIDAT ČLENA“.
6) Systém odešle data na backend, ověří jejich unikátnost a správnost a pokud je vše v pořádku, uloží nového člena do databáze.
7) Systém automaticky vygeneruje novému členovi bezpečné heslo a zašle mu jej na zadaný email.
8) Systém zobrazí uživateli potvrzení, že přidání člena proběhlo v pořádku.
9) Po potvrzení hlášky je uživatel přesměrován na stránku „Můj Tým“.

Alternative flow:

1) Systém při ověřování (krok 6 v main flow) zjistí, že zadaný e-mail či telefon (u hráče případně ještě FAČR ID) již v databázi existuje, nebo že data nemají správný formát.
2) Systém odmítne uložení a vrátí chybovou hlášku, kterou frontend zachytí a uživateli zobrazí detailní vysvětlení chyby.
3) Registrace je přerušena, data zůstávají vyplněná ve formuláři pro možnost opravy.

<u>BPMN</u>



3) Funkční a nefunkční požadavky

Funkční požadavky:

Správa uživatelů a rolí:

- Systém musí umožnit zabezpečené přihlášení a odhlášení uživatele.
- Systém musí rozlišovat uživatelské role (např. Trenér, Hráč) a na jejich základě řídit přístupová oprávnění.
- Systém musí uživateli s rolí Trenér umožnit přidávat nové uživatele, aktualizovat jejich údaje (jméno, příjmení, telefon, email, město) a deaktivovat jejich účty.
- Systém nesmí umožnit fyzické smazání uživatele z databáze z důvodu zachování datové integrity, pouze jeho deaktivaci (tzv. soft delete).
- Systém musí umožnit evidenci a zobrazení zdravotního stavu hráče (např. aktivní, zraněný).
- Systém musí umožnit uložení a zobrazení kontaktních údajů (telefon, e-mail) na hráče, případně na jejich zákonné zástupce.

Aktivity a docházka:

- Systém musí umožnit Trenérovi vytvářet a spravovat týmové aktivity (tréninky, zápasy) s definicí času a místa konání.
- Systém musí uživatelům zobrazit přehled naplánovaných aktivit v seznamové formě.
- Systém musí Hráčům umožnit evidovat svou účast/neúčast na aktivitě (omluvenka), avšak tato možnost se musí automaticky uzamknout 12 hodin před plánovaným začátkem aktivity.
- Systém musí v případě uzamčené docházky (méně než 12 hodin do začátku) zobrazit Hráči informaci, že pro pozdní omluvu z důvodu náhle události musí kontaktovat Trenéra napřímo.
- Systém nesmí toto časové uzamčení aplikovat na uživatele s rolí Trenér, který má právo docházku i statistiky doplňovat a upravovat kdykoliv (před i po skončení aktivity).

Statistiky a výkon:

- Systém musí uživateli s rolí Trenér umožnit záznam individuálních i týmových statistik u jednotlivých aktivit (např. góly, asistence, odehraný čas).
- Systém musí generovat historický přehled výkonů a docházky pro jednotlivé hráče i celý tým.

Notifikace a chybové stavy:

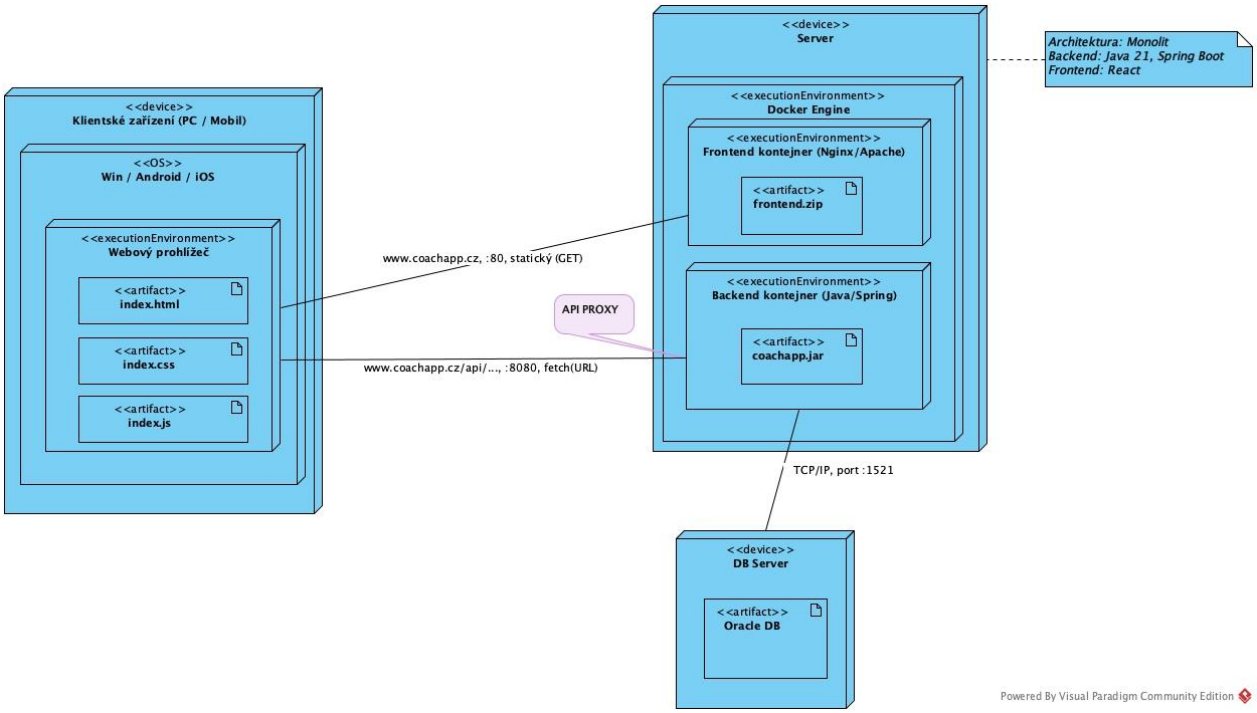
- Systém musí automaticky upozornit uživatele (primárně e-mailem, volitelně notifikací v aplikaci) na vytvoření nové aktivity nebo změnu v existující aktivitě.
- Systém musí ošetřit neoprávněný přístup (chyba 403) a přístup na neexistující adresy (chyba 404) přesměrováním na uživatelsky přívětivou chybovou stránku.

Nefunkční požadavky:

- Doba odezvy systému a načtení běžné stránky nesmí přesáhnout 2 sekundy při zátěži do 50 současně přistupujících uživatelů za standardních podmínek provozu (tj. na serveru o minimálním výkonu 2CPU jader a 4 GB RAM).
- Systém musí vykazovat dostupnost minimálně 99 % času (vyjma plánovaných servisních odstávek).
- Data v databázi musí být automaticky zálohována v pravidelných denních intervalech pro možnost obnovy při selhání.
- Systém musí využívat relační databázi s nastavenými cizími klíči pro zajištění referenční integrity.
- Hesla uživatelů musí být před uložením do databáze bezpečně zahashována (algoritmus BCrypt) a veškerý přenos dat musí pro zajištění bezpečnosti probíhat přes šifrovaný protokol HTTPS.
- Klientská část (frontend) bude realizována pomocí knihovny React, serverová část (backend) v jazyce Java.
- Aplikace musí být responzivní (plně funkční v PC i mobilních prohlížečích).
- Aplikace bude primárně lokalizovaná do českého jazyka.

SOFTWARE DESIGN DESCRIPTION

4) UML Deployment diagram

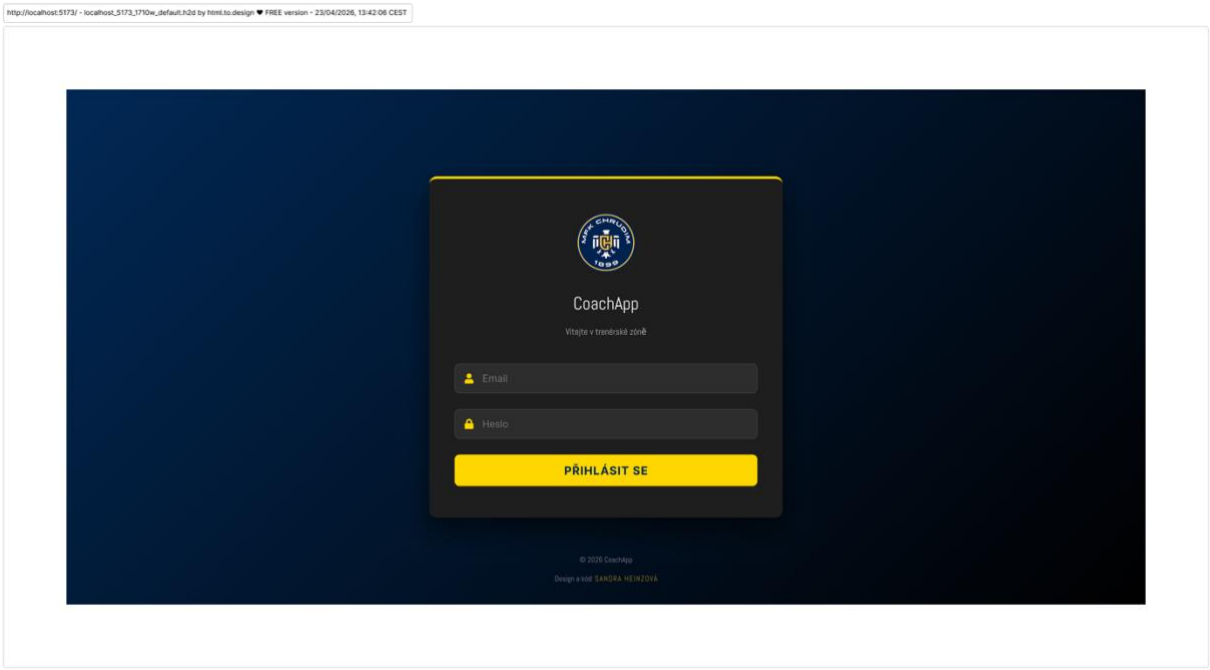


5) Návrh UI

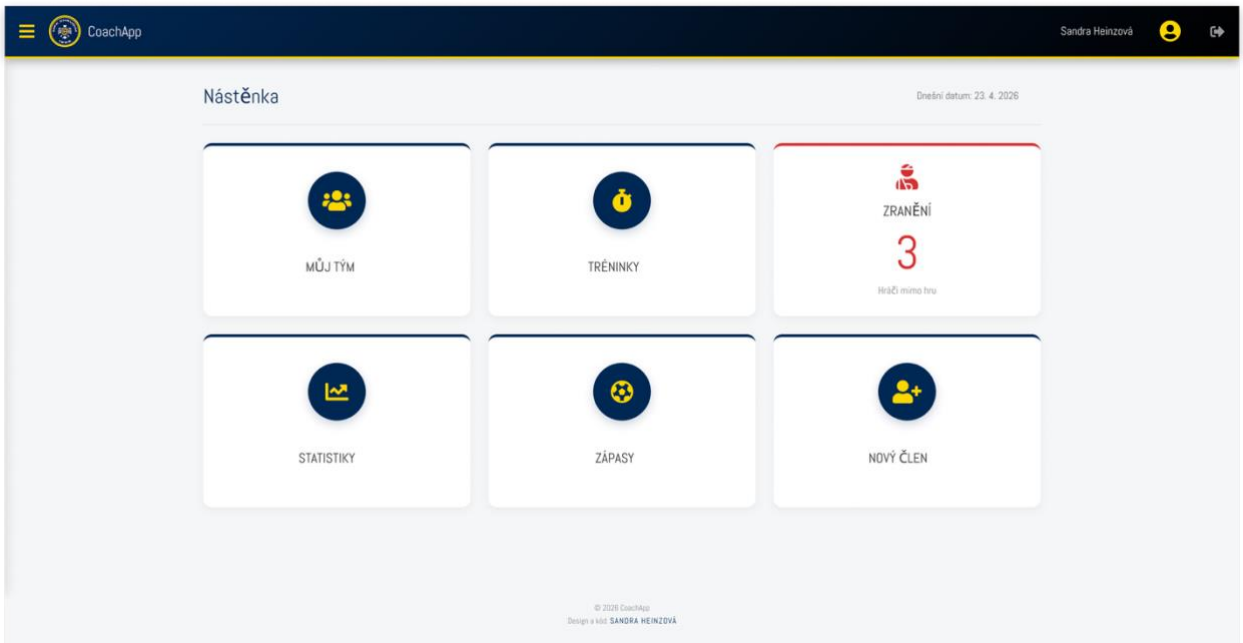
Pro vizuální návrh aplikace byl vytvořen jednotný design systém, který zajišťuje konzistenci napříč všemi obrazovkami. Návrhy byly zpracovány v nástroji Figma.
Základem aplikace je tmavě modrá barva, která tvoří hlavičky a hlavní navigační prvky. Jako kontrastní sekundární barva byla zvolena žlutá, která je vyhrazena pro hlavní interakční prvky. Pro okamžitou zpětnou vazbu uživateli jsou využity sémantické barvy – zelená pro aktivní stavy a červená pro destruktivní akce (zrušení) nebo chybová hlášení.
Velký důraz byl kladen na dostatečný kontrast textu vůči pozadí z hlediska přístupnosti. Velikosti písma jsou systematicky odstupňovány (např. u modálních oken), aby uživatel na první pohled rozeznal hlavní nadpis, podkategorie a běžná data, čímž se zrychluje orientace v rozhraní.
Základním stavebním kamenem rozhraní jsou zaoblené "karty", které přehledně shlukují související informace k danému tréninku či zápasu. Karta tak působí jako jeden uchopitelný prvek.
Pro komplexnější akce, jako je tvorba zápasové soupisky, jsou navržena modální (vyskakovací) okna. Toto specifické UX rozhodnutí bylo učiněno za účelem zachování kontextu – uživatel díky tomu nepřichází o aktuální stav na pozadí, vyhne se zbytečnému přesměrování a načítání nových stránek, což výrazně snižuje kognitivní zátěž. Modální okna

využívají poloprůhledné ztmavení pozadí, které efektivně udržuje pozornost uživatele na probíhajícím úkolu a vizuálně odděluje formulář od zbytku aplikace.

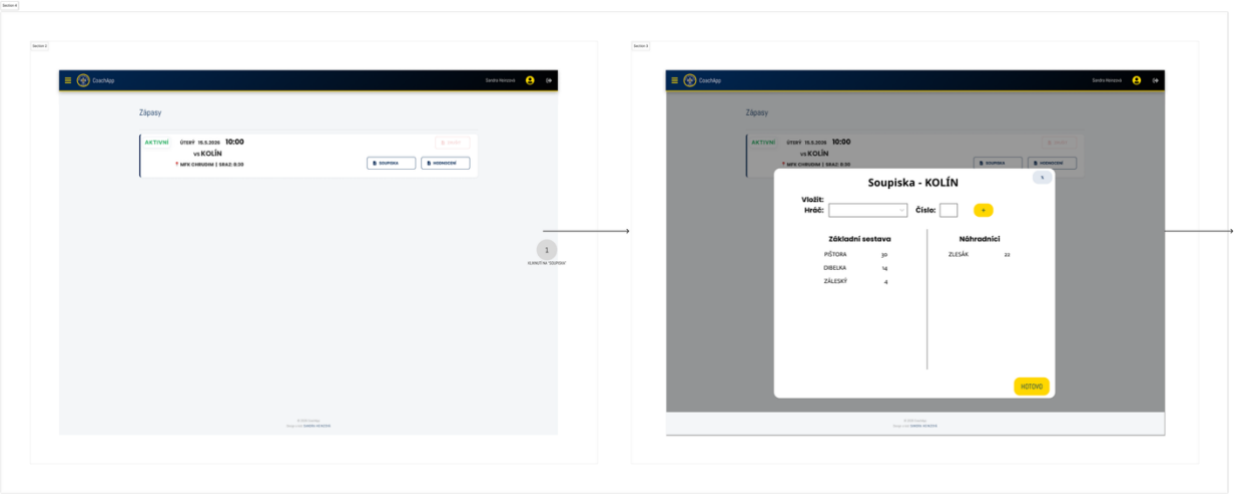
Obr. 1 – Přihlašovací login stránka



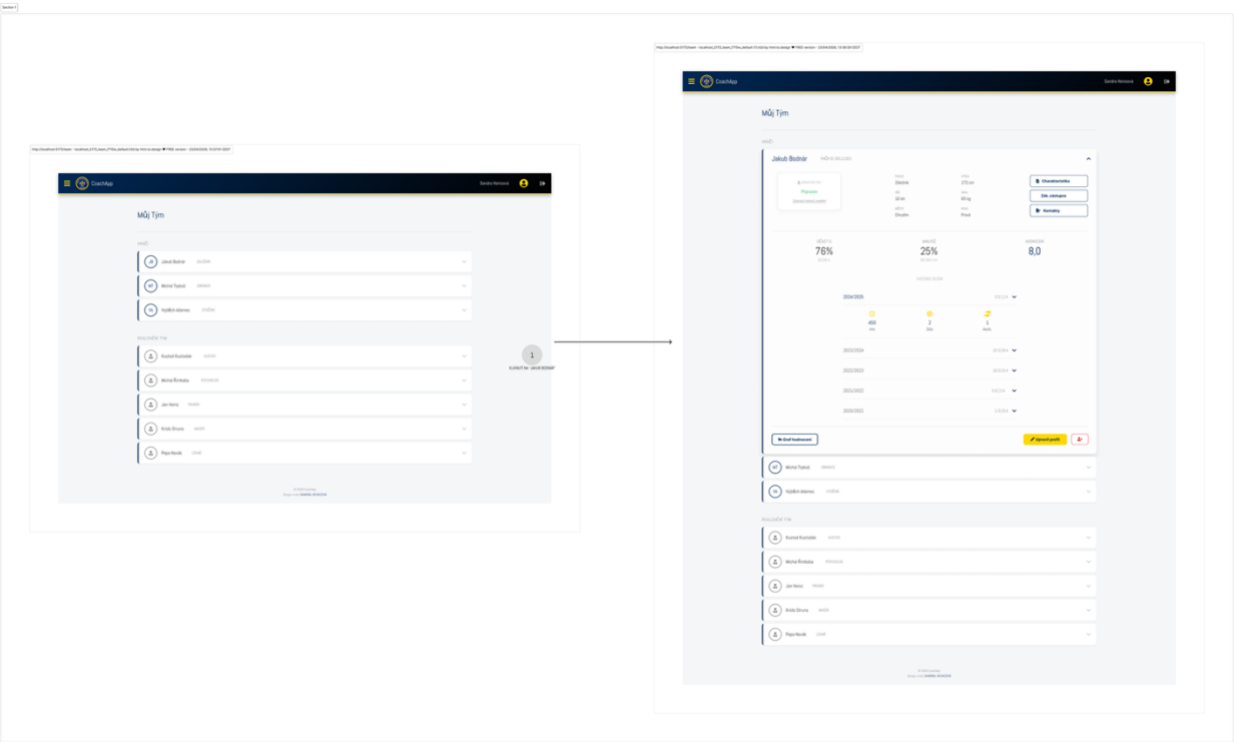
Obr. 2 – Nástěnka



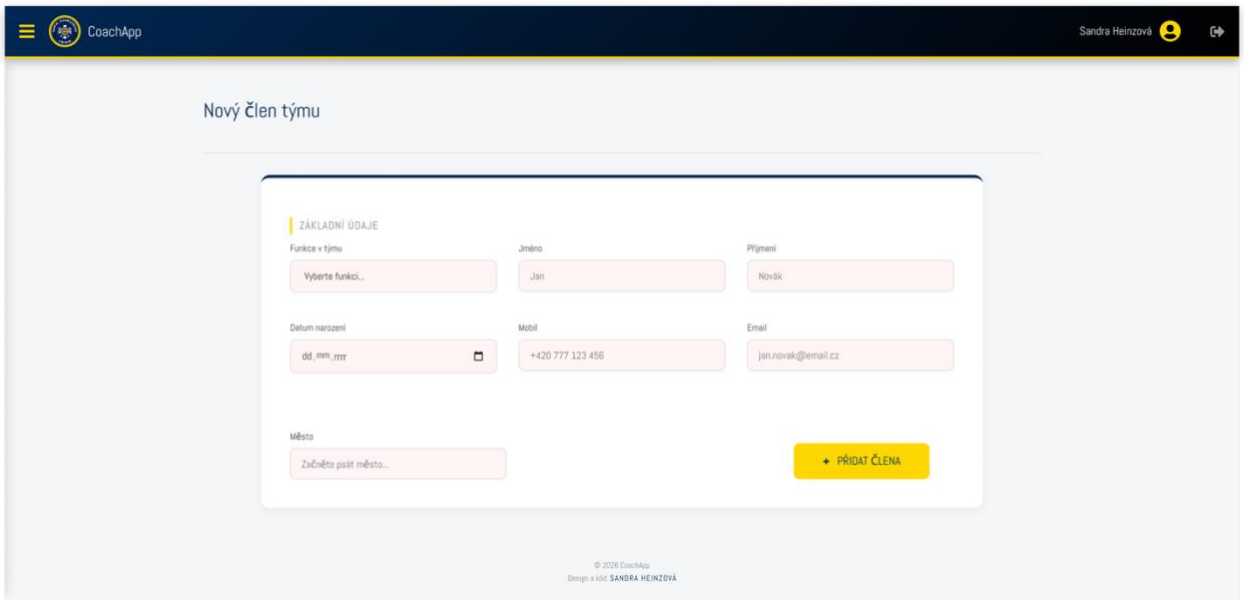
Obr. 3 - Zápasy



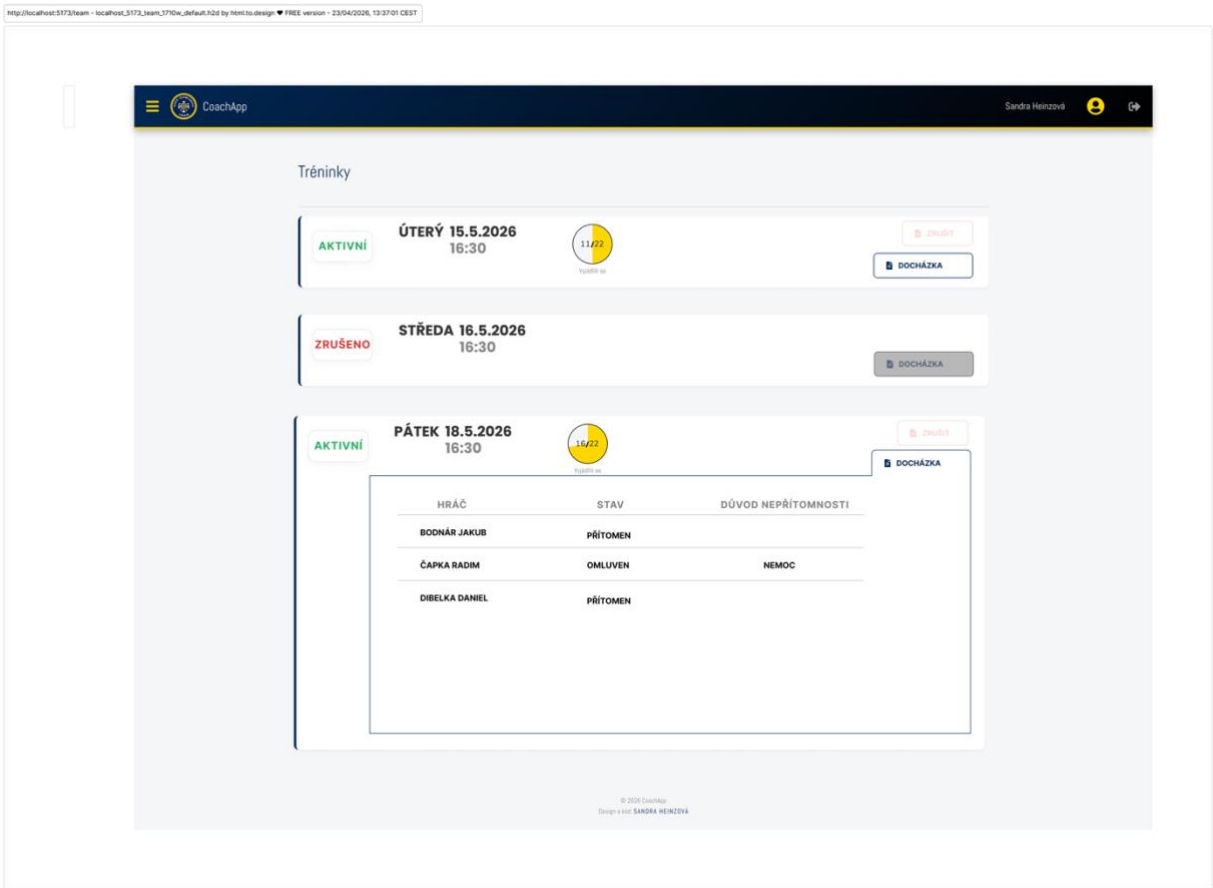
Obr. 4 – Můj tým



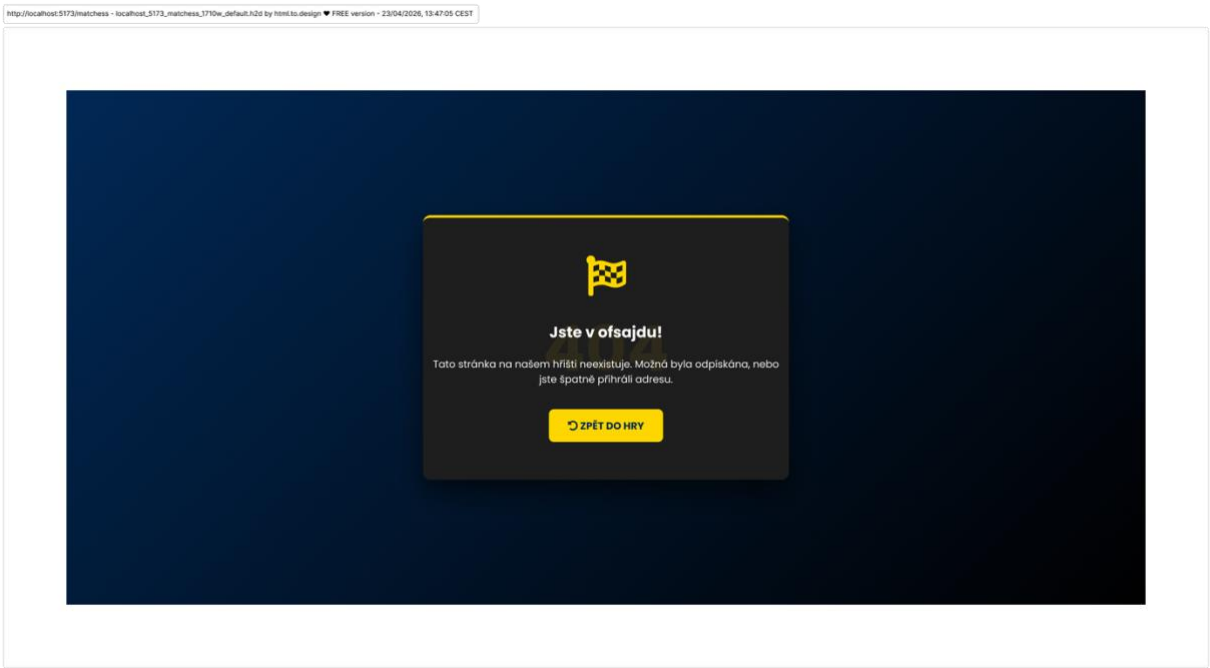
Obr. 5 – Nový člen týmu



Obr. 6. - Tréninky



Obr. 7–404 Stránka nenalezena



6) Návrh API

Registrace nového člena

- url adresa - /api/auth/register
- metoda POST
- Payload povinné – jméno, příjmení, email, ID role
- Response – 201 Created (Člen vytvořen)
  – 400 Bad request (chyba validace dat)

Přihlášení uživatele

- url adresa - /api/auth/login
- metoda POST
- Payload povinné – email, heslo
- Response – 200 OK (Uživatel přihlášen)
  – 401 Unauthorized (chyba validace dat)
  – 500 Internal server error (chyba při načítání profilu či databáze)

Získání seznamu všech uživatelů

- url adresa - /api/users
- metoda GET
- Payload – žádný
- Response – 200 OK (Vrátí seznam všech uživatelů)

Získání detailu uživatele

- url adresa - /api/users/{id}
- metoda GET
- Payload – žádný
- Response – 200 OK (Vrátí profil konkrétního uživatele)
  – 404 Not Found (Uživatel nenalezen)

Aktualizace údajů uživatele

- url adresa - /api/users/{id}
- metoda PUT
- Payload nepovinné – email, telefon, město
- Response – 200 OK (Údaje byly úspěšně aktualizovány)
  – 500 Internal server error (Chyba při aktualizaci dat)

Deaktivace uživatele

- url adresa - /api/users/{id}/deactivate
- metoda PATCH
- Payload – žádný
- Response – 200 OK (Člen byl deaktivován)
  – 500 Internal server error (Chyba při deaktivaci)

Získání seznamu všech tréninků

- url adresa - /api/trainings
- metoda GET
- Payload – žádný
- Response – 200 OK (Vrátí seznam všech tréninků)

Získání seznamu všech zápasů

- url adresa - /api/matches
- metoda GET
- Payload – žádný
- Response – 200 OK (Vrátí seznam všech zápasů)

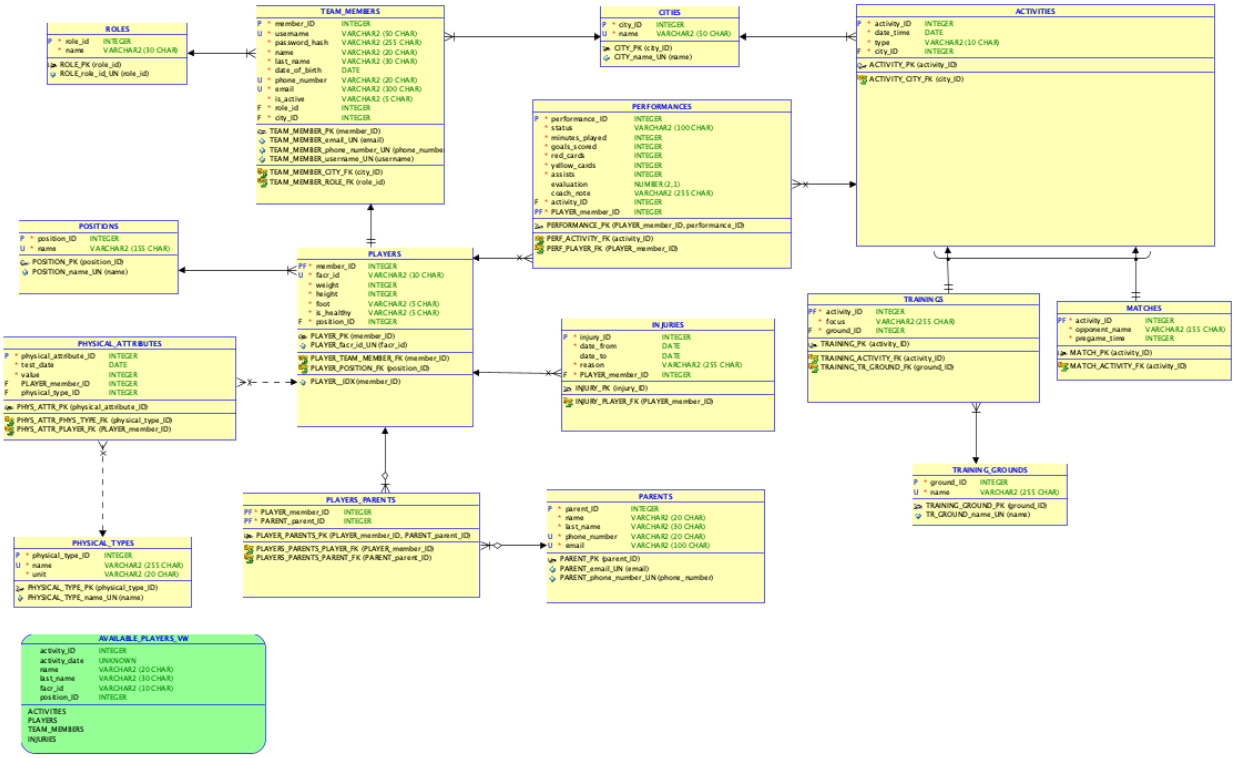
Přidání nového tréninku

- url adresa - /api/trainings
- metoda POST
- Payload povinné – datum, čas, místo
- Response – 201 Created (Trénink úspěšně vytvořen)
  – 400 Bad request (Chyba validace dat)

Přidání nového zápasu

- url adresa - /api/matches
- metoda POST
- Payload povinné – datum, čas, soupeř, místo
- Response – 201 Created (Zápas úspěšně vytvořen)
  – 400 Bad request (Chyba validace dat)

7) Návrh databáze



8) Návrh backendu

Backendová část aplikace je navržena v programovacím jazyce Java s využitím moderního frameworku Spring Boot. Tento framework byl zvolen pro svou robustnost, rychlost vývoje a snadnou tvorbu RESTových endpointů. O komplexní zabezpečení aplikace se stará modul Spring Security. Systém komunikuje s relační databází pomocí objektově-relačního mapování.

Proces autentizace slouží k jednoznačnému ověření identity uživatele – v našem případě k tomu dochází zadáním platného e-mailu a hesla. Pro udržení stavu přihlášeného uživatele je využívána stavová architektura pomocí relací (Sessions). Po úspěšném ověření identity server vytvoří na své straně unikátní relaci a klientovi (prohlížeči) odešle identifikátor této relace v podobě cookies s názvem JSESSIONID. Tento identifikátor je následně automaticky přikládán ke každému dalšímu http požadavku, což serveru umožňuje rozpoznat již přihlášeného uživatele po celou dobu jeho aktivity v aplikaci.

Systém využívá model RBAC (Role-Based Access Control) a podporuje libovolné množství rolí (např. Trenér, Hráč, Lékař). Oprávnění jsou však striktně oddělena – plnou správu systému a členů mají uživatelé s oprávněním **Admin** a **Trenér**, zatímco ostatní role (např. **Hráč**) mají oprávnění omezeno pouze na čtení a úpravu vlastního profilu.

Modul Spring Security na základě těchto rolí automaticky blokuje neoprávněné přístupy k chráněným endpointům.
Veškerá komunikace mezi klientem a serverem je šifrována pomocí protokolu HTTPS.
Bezpečné spojení je navazováno pomocí kombinace kryptografických metod:

- Asymetrické šifrování: slouží k bezpečnému ustavení spojení. Server se klientovi prokáže certifikátem od důvěryhodné autority za použití páru veřejného a soukromého klíče.
- Symetrické šifrování: Jakmile je dohodnut společný tajný klíč, samotný přenos dat probíhá pomocí rychlého symetrického šifrování.